

Desplegar y conducir

Una vez que hayas verificado el hardware y registrado los *offsets*, conecta YAGSL con el código de tu robot. SwerveParser lee el directorio de configuración swerve/base y crea automáticamente un SwerveDrive de YAMS. Tu solo necesitas programar el subsistema y las asignaciones de controles que lo rodean.

Crear un subsistema swerve.

None

```
import edu.wpi.first.math.geometry.Pose2d;
import edu.wpi.first.math.geometry.Rotation2d;
import edu.wpi.first.math.kinematics.ChassisSpeeds;
import edu.wpi.first.wpilibj.Filesystem;
import edu.wpi.first.wpilibj2.command.Command;
import edu.wpi.first.wpilibj2.command.SubsystemBase;
import java.io.File;
import java.util.function.DoubleSupplier;
import swervelib.parser.SwerveParser;
import yams.mechanisms.config.SwerveDriveConfig;
import yams.mechanisms.swerve.SwerveDrive;
import yams.mechanisms.swerve.utility.SwerveInputStream;
import yams.motorcontrollers.SmartMotorControllerConfig.TelemetryVerbosity;

public class SwerveDriveSubsystem extends SubsystemBase
{

    private SwerveDrive drive;

    public SwerveDriveSubsystem()
    {
        var cfg = new SwerveDriveConfig()
            .withStartingPose(new Pose2d(3, 3, Rotation2d.kZero))
            .withSubsystem(this)
            .withTelemetry(TelemetryVerbosity.HIGH);
        try
        {
            drive = new SwerveParser(new File(Filesystem.getDeployDirectory(),
"swerve/base"))
                .createSwerveDrive(cfg);
        } catch (Exception e)
        {
            throw new RuntimeException(e);
        }
    }
}
```

```

    }

    public SwerveInputStream getAngularVelocityStream(DoubleSupplier x,
DoubleSupplier y,
                                                    DoubleSupplier rot)
    {
        return new SwerveInputStream(drive, x, y, rot);
    }

    public Command drive(SwerveInputStream stream)
    {
        return drive.drive(() -> ChassisSpeeds.fromFieldRelativeSpeeds(stream.get(),
                                                                    new
Rotation2d(drive.getGyroAngle())));
    }

    /** Pon a cero la orientacion del giroscopico. Asigna esto a una combinacion de
botones para poder recuperarlo durante un partido. */
    public Command zeroGyro()
    {
        return runOnce(() -> drive.zeroGyro());
    }

    @Override
    public void periodic()
    {
        drive.updateTelemetry();
    }

    @Override
    public void simulationPeriodic()
    {
        drive.simIterate();
    }
}

```

SwerveParser lee swervedrive.json y todos los archivos a los que este hace referencia dentro de modules/. Después, identifica el hardware mediante la librería correspondiente de cada fabricante y devuelve un SwerveDrive de YAMS completamente configurado. De esta manera, no tienes que mantener manualmente un archivo de constantes.

TelemetryVerbosity.HIGH resulta útil durante la puesta en marcha del robot, ya que genera los valores del panel utilizados en el paso 4. Sin embargo, también aumenta el tráfico de

NetworkTables, así que conviene reducir el nivel de telemetría una vez que hayas terminado de ajustar el sistema.

Configurar el controlador.

None

```
import edu.wpi.first.wpilibj2.command.button.CommandXboxController;
import frc.robot.subsystems.swervedrive.SwerveDriveSubsystem;
import yams.mechanisms.swerve.utility.SwerveInputStream;

public class RobotContainer
{

    final CommandXboxController driverXbox = new CommandXboxController(0);

    private final SwerveDriveSubsystem swerve = new SwerveDriveSubsystem();

    private final SwerveInputStream driveAngularVelocity =
        swerve.getAngularVelocityStream(
            driverXbox::getLeftY,
            driverXbox::getLeftX,
            () -> driverXbox.getRawAxis(2))
            .withAllianceRelativeControl();

    public RobotContainer()
    {
        configureBindings();
    }

    private void configureBindings()
    {
        // Comando de conduccion predeterminado
        swerve.setDefaultCommand(swerve.drive(driveAngularVelocity));

        // Poner a cero el giroscopico con Start + Back.
        // Usalo si la orientacion relativo al campo empieza a desviarse.
        driverXbox.start().and(driverXbox.back()).onTrue(swerve.zeroGyro());
    }
}
```

¿Por qué asignar zeroGyro() a una combinación de botones? Los giroscópicos pueden desviarse con el tiempo o encenderse cuando el robot está orientado en una dirección incorrecta. Una combinación de botones, como Start +

Back o ambos bumpers, permite que el conductor vuelva a alinear instantáneamente el control relativo al campo sin tener que tocar la Driver Station.

Conviene asignar siempre esta función a algún control. Puede salvar un partido cuando el giroscopio pierde su referencia.

Primera prueba de conducción

1. Despliega el código en el robot.
2. Con el robot elevado sobre soportes o bloques, habilitarlo en modo Teleop y empuja suavemente hacia delante el joystick de movimiento., Las cuatro ruedas debe orientarse en la misma dirección y el robot debe intentar desplazarse hacia delante.
3. Utiliza el eje de rotación; el robot debe girar alrededor de su propio centro, sin desplazarse lateralmente mientras rota.
4. Si uno de los módulos se opone a los demás, por ejemplo, gira hacia un ángulo incorrecto o mueve la rueda en sentido contrario, vuelve al [paso 4](#) de configuración y revisa la inversión y el offset de este módulo.
5. Cuando el robot funcione correctamente mientras está elevado, Pruébalo en el suelo a baja velocidad antes de aumentar la velocidad o la potencia.

Felicidades!!!!

Ya tienes un robot swerve capaz de conducir correctamente. A partir de aquí, consulta la sección How-To Guides, para aprender a [ajustar las ganancias PIDE](#), diagnosticar problemas de desviación y realizar otras tareas de configuración a las que probablemente tendrás que volver durante la temporada de FRC.